

Enterprise REST API Design & Governance Guide

1. Introduction

This guide defines **enterprise-grade standards and best practices** for designing, building, securing, operating, and governing REST APIs. It is intended for architects, backend engineers, platform teams, and API governance bodies to ensure APIs are **consistent, secure, scalable, observable, and compliant** across the organization.

2. REST API Design Principles (Enterprise-Grade)

Core Principles

- **Resource-Oriented Design:** APIs expose business resources, not actions
- **Statelessness:** Each request contains all necessary context
- **Consistency:** Uniform patterns across all APIs
- **Consumer-First Design:** Optimize for developer experience
- **Loose Coupling:** Avoid client dependency on internal implementation
- **Evolvability:** APIs must support backward-compatible evolution

Key Outcomes

- Predictable APIs
 - Faster consumer adoption
 - Reduced breaking changes
 - Long-term maintainability
-

3. Resource & URI Naming Standards

URI Rules

- Use **nouns**, not verbs
- Use **plural** resource names
- Use **lowercase** with hyphens
- Represent hierarchy via path segments

Examples

```
GET /api/v1/orders
GET /api/v1/orders/{orderId}/items
```

Anti-Patterns

```
POST /createOrder
GET /getUserDetails
```

4. HTTP Methods & Status Codes

Method Semantics

Method	Purpose
GET	Retrieve resource(s)
POST	Create resource
PUT	Replace resource
PATCH	Partial update
DELETE	Remove resource

Status Codes

- **2xx** – Success (200, 201, 204)
- **4xx** – Client errors (400, 401, 403, 404, 409)
- **5xx** – Server errors (500, 503)

5. Request & Response Schemas and Headers

Request Standards

- JSON payloads only
- Explicit required vs optional fields
- Validation at API boundary

Common Headers

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer <token>
X-Correlation-Id: <uuid>
```

Response Envelope (Recommended)

```
{
  "data": {},
  "meta": {},
  "links": {}
}
```

6. Standard Error Model

Error Response Format

```
{
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "Order does not exist",
    "details": "No order found for id 123",
    "traceId": "abc-123"
  }
}
```

Error Guidelines

- Machine-readable error codes
- Human-readable messages
- Traceability via correlation IDs

7. Pagination, Filtering & Sorting

Pagination

- Offset-based: `page`, `size`
- Cursor-based for large datasets

```
GET /orders?page=1&size=20
```

Filtering & Sorting

```
GET /orders?status=SHIPPED&sort=createdAt,desc
```

8. Authentication & Authorization (OAuth2, JWT)

Authentication

- OAuth2 (Authorization Code, Client Credentials)
- JWT access tokens

Authorization

- Scopes for coarse-grained access
- Roles / ABAC for fine-grained control

JWT Best Practices

- Short-lived access tokens
- Validate issuer, audience, expiry

9. Security Best Practices (OWASP API Top 10)

Mandatory Controls

- Strong authentication
- Rate limiting & throttling
- Input validation
- Schema validation
- Least privilege access

Additional Measures

- API Gateway enforcement
- Secrets management
- Encryption at rest & in transit

10. Versioning & Backward Compatibility

Versioning Strategies

- URI-based: `/v1`, `/v2`
- Header-based (advanced use cases)

Compatibility Rules

- Never break existing fields
- Only add optional fields

- Deprecate before removal
-

11. Idempotency

When Required

- POST operations
- Payment & order creation

Idempotency Key

Idempotency-Key: <uuid>

Ensures safe retries without duplicate processing.

12. OpenAPI / Swagger Documentation Standards

Requirements

- OpenAPI 3.x specification
- Complete schemas
- Request/response examples
- Error models documented

Benefits

- Contract-first development
 - Client SDK generation
 - Automated validation
-

13. Logging, Monitoring & Observability

Logging

- Structured JSON logs
- Correlation IDs

Metrics

- Latency
- Error rates
- Throughput

Tracing

- Distributed tracing (OpenTelemetry)
-

14. Performance & Scalability

Techniques

- HTTP caching
- Async processing
- Pagination limits
- Horizontal scaling

API Gateway Responsibilities

- Caching
 - Rate limiting
 - Load balancing
-

15. Testing Standards

Testing Pyramid

- Unit tests
- Integration tests
- Contract tests

Contract Testing

- Prevent breaking changes
 - Validate backward compatibility
-

16. API Governance & Lifecycle Management

Lifecycle Stages

- Design
- Develop
- Test
- Release
- Operate
- Retire

Governance Controls

- Design reviews
 - Schema validation
 - Promotion workflows
-

17. Compliance & Audit Readiness

Key Areas

- Audit logs
- Access traceability
- Data retention policies
- Regulatory compliance (HIPAA, GDPR, SOC2)

Best Practices

- Immutable logs
 - Regular audits
 - Clear ownership
-

18. Conclusion

Following this guide ensures APIs are **secure, scalable, observable, and enterprise-ready**, while enabling teams to move fast without sacrificing quality or governance.